Chapitre 4

# Communication asynchrone dans les applications réparties

# Pourquoi l'asynchronisme

La communication asynchrone joue un rôle central dans les applications réparties modernes. Contrairement aux interactions synchrones, l'asynchronisme propose un modèle d'échange où les processus n'ont pas besoin d'attendre une réponse immédiate pour continuer leur exécution.

→ Ce modèle répond aux défis de la distribution, de la scalabilité et de la tolérance aux pannes.

- L'émetteur n'attend pas la fin du traitement.
- Il envoie et continue son exécution.
- Le retour (si nécessaire) arrive plus tard, via callback ou message de réponse.

- ✓ le message peut être stocké,
- ✓ transmis plus tard,
- ✓ ré-essayé,
- ✓ consommé par plusieurs acteurs.

→ On parle de découplage temporel.

L'asynchronisme est la base des architectures : orientées événements, basées sur des brokers de messages, serverless avec déclencheurs, microservices distribués.

# Les intervenants principaux

## ★ Le producteur :

C'est l'entité (service, processus, module, application…) qui :

- ✓ génère un message, un événement ou une tâche,
- ✓ l'envoie vers un système intermédiaire sans attendre la réponse du consommateur,
- ✓ continue son exécution immédiatement.

➔ Le producteur est découplé du traitement.

## ★ Le consommateur :

C'est l'entité qui :

- ✓ reçoit le message ou l'événement,
- ✓ le traite plus tard, selon sa disponibilité,
- ✓ peut être unique ou multiple (selon le modèle point-à-point ou publish/subscribe).

➔ Le consommateur est découplé à la fois dans le temps et dans l'espace du producteur.

# Les intervenants principaux

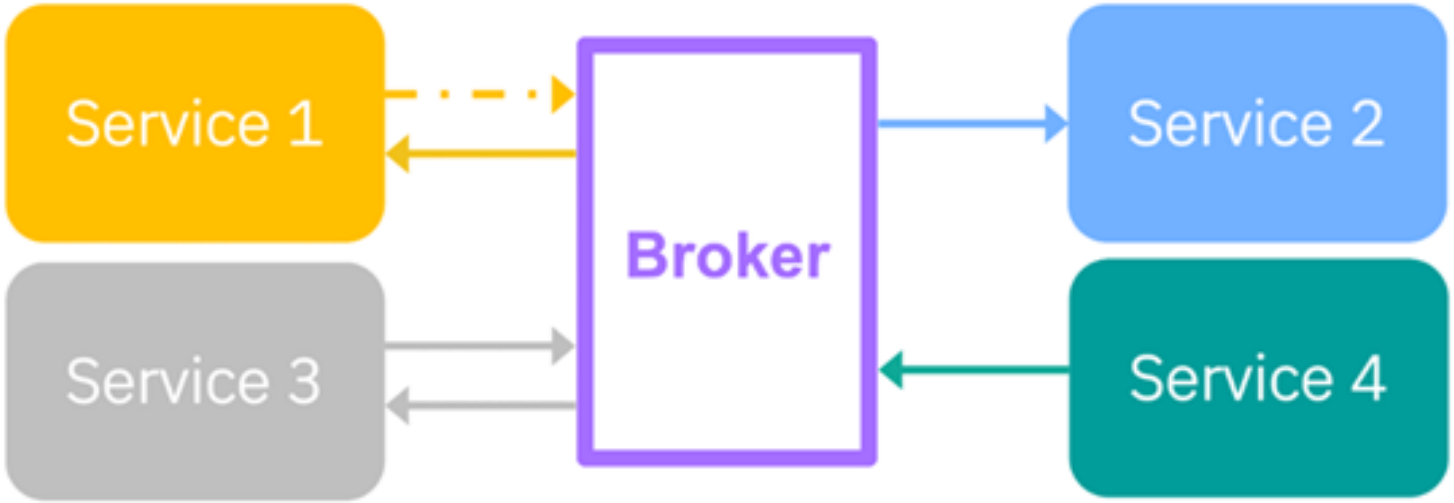
## ★ L'Intermédiaire (Broker / Queue / Middleware de Messages)

Cet intermédiaire :

- ✓ stocke les messages,
- ✓ les achemine,
- ✓ garantit la livraison,
- ✓ gère les pics de charge.

C'est l'intermédiaire qui rend possible :

- ➢ le découplage temporel,
- ➢ la résilience,
- ➢ la scalabilité.



# Motivation

## 1. Eviter le blocage

L'asynchrone permet de ne pas bloquer l'exécution, de continuer d'autres tâches pendant l'attente.

→ Cela augmente la réactivité globale.

## 2. Découpler les producteurs et consommateurs

- le producteur envoie un message, événement ou tâche,
- le consommateur la traite plus tard, au moment qui lui convient,
- ils n'ont pas besoin d'être actifs en même temps.

→ Cela permet l'ajout/suppression de consommateurs sans impacter les producteurs et la variation de charge sans saturation immédiate.

## 2. Améliorer la scalabilité

L'asynchronisme facilite la montée en charge : Le système absorbe les pics d'activité via des files/messages.

# Principes clés de la communication asynchrone

**Découplage temporel** : L'émetteur et le récepteur n'ont plus l'obligation d'agir au même moment.

**Découplage spatial :** Les acteurs n'ont pas nécessairement connaissance l'un de l'autre.

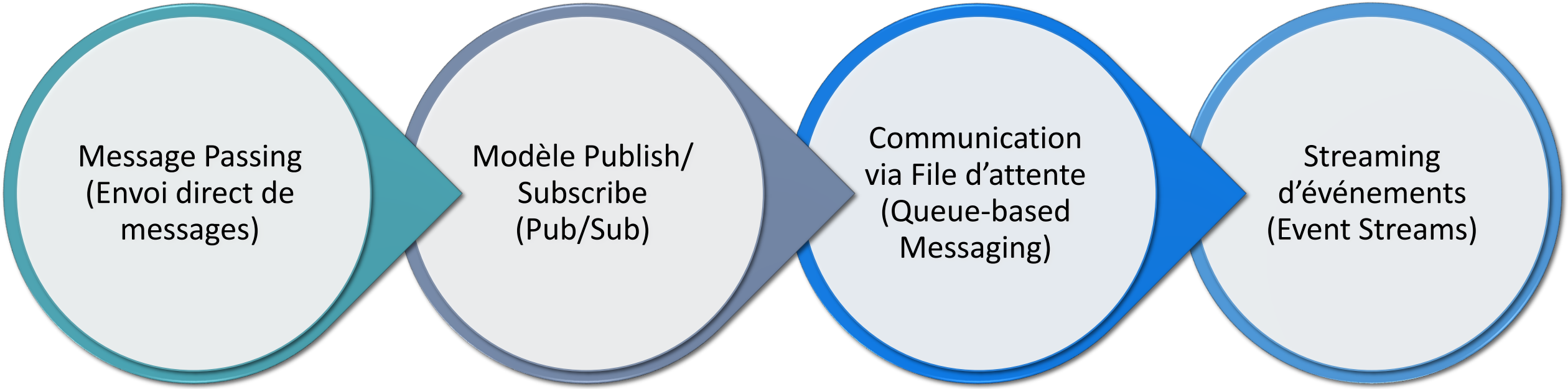
**Résilience et tolérance aux pannes :** Le système continue de fonctionner même si un service tombe en panne, car les messages restent stockés jusqu'au redémarrage.

**Evolutivité (scalabilité) :** De nouveaux consommateurs peuvent être ajoutés facilement.

**Gestion des pics de charge :** Lorsque la consommation est plus lente que la production, le système régule le flux en ralentissant, mettant en file ou rejetant proprement des messages.

# Modèles de la communication asynchrone

Dans les systèmes distribués, plusieurs modèles permettent de mettre en œuvre une communication asynchrone. Ils se distinguent par la nature des échanges, le rôle des intermédiaires et les garanties offertes.



# Envoi direct de message

Le Message Passing est un modèle de communication asynchrone où un service (émetteur) envoie un message à un autre service (récepteur) sans attendre de réponse immédiate.




→ Ce modèle est utilisé dans : les architectures réactives et les microservices.

★ **Caractéristiques principales :**

- Communication point-à-point : Le message a un destinataire unique. → **Couplage spatial**
- Asynchronisme : L'émetteur n'attend pas la réponse : il continue son exécution immédiatement.
- Message indépendant du protocole : Le message peut être envoyé via webSocket, socket réseau, HTTP asynchrone, gRPC asynchrone, middelware, …

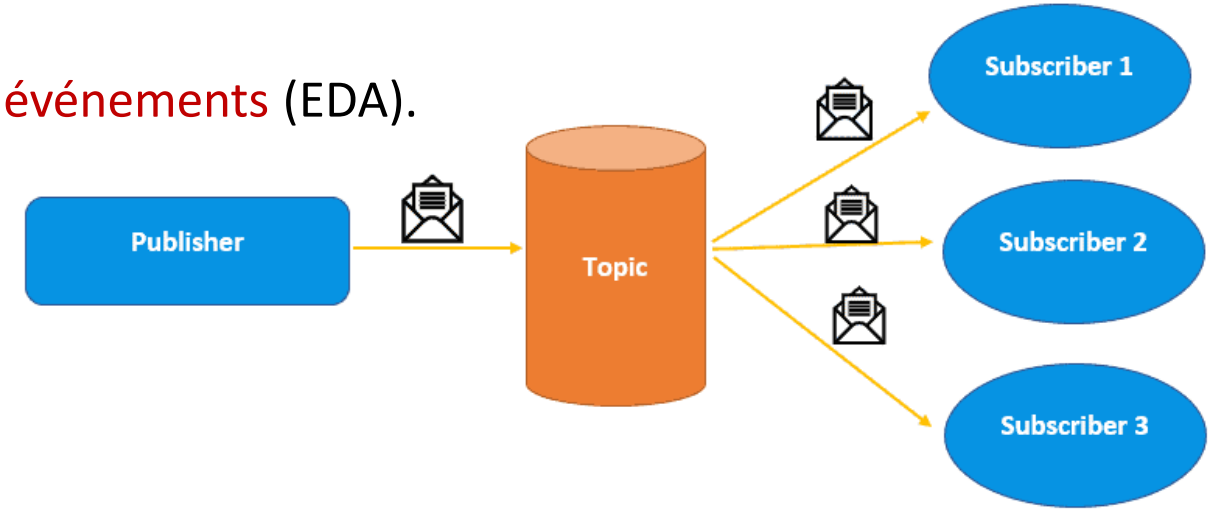
⚠ Si le récepteur est indisponible, le message peut être perdu

Technologies majeures :
- RabbitMQ
- NATS
- Microsoft MSMQ

# Modèle Publish / Subscribe

Le modèle Publish/Subscribe est un modèle de communication asynchrone dans lequel les producteurs (publishers) publient les messages dans un intermédiaire (souvent appelé broker, canal ou topic).
Les abonnés (subscribers) déclarent leur intérêt pour certains types de messages auprès de cet intermédiaire.

→ Ce modèle est Idéal pour les architectures orientées événements (EDA).




★ **Caractéristiques principales :**

- ➢ Découplage fort : spatial, temporel et logique.
- ➢ Scalabilité.
- ➢ Résilience.
- ➢ Diffusion efficace (broadcast).

★ **Deux types principaux de Pub/Sub :**

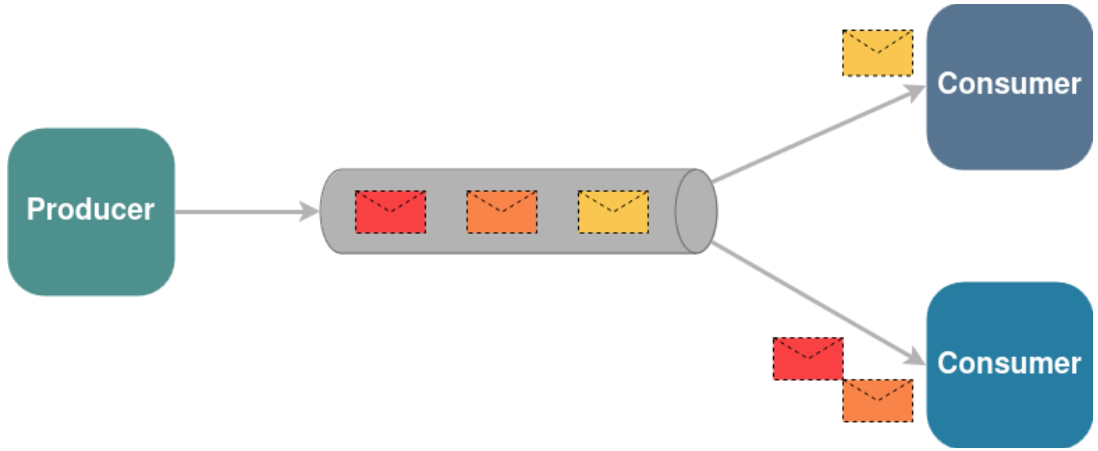
- ✓ Pub/Sub basé sur les topics (Topic-based) où les subscribers choisissent les topics. → Les messages sont publiés dans des topics nommés.
- ✓ Pub/Sub basé sur le contenu (Content-based) où les abonnés spécifient des filtres. → Le routage dépend du contenu du message.

Technologies majeures :

- Redis Pub/Sub
- Google Pub/Sub
- Apache Kafka

# Communication Via File d'attente

Les systèmes basés sur des files, ou Queue-based Messaging, sont un modèle dans lequel un message envoyé par un producteur (sender) est placé dans une file d'attente (queue) gérée par un message broker.
Le message reste dans la file jusqu'à ce qu'un consommateur (receiver) le récupère et le traite.




→ Dans ce modèle, chaque message est consommé par un seul destinataire.

La queue joue le rôle de mémoire temporaire (si le receiver tombe en panne) et d'un système de régulation (absorbe les pics de charge)

Le sender ne connaît pas le receiver (ni son adresse, ni son état). Il connaît juste : l'adresse du broker et le nom de la queue. → C'est le broker qui s'occupe de transférer le message au bon consommateur.

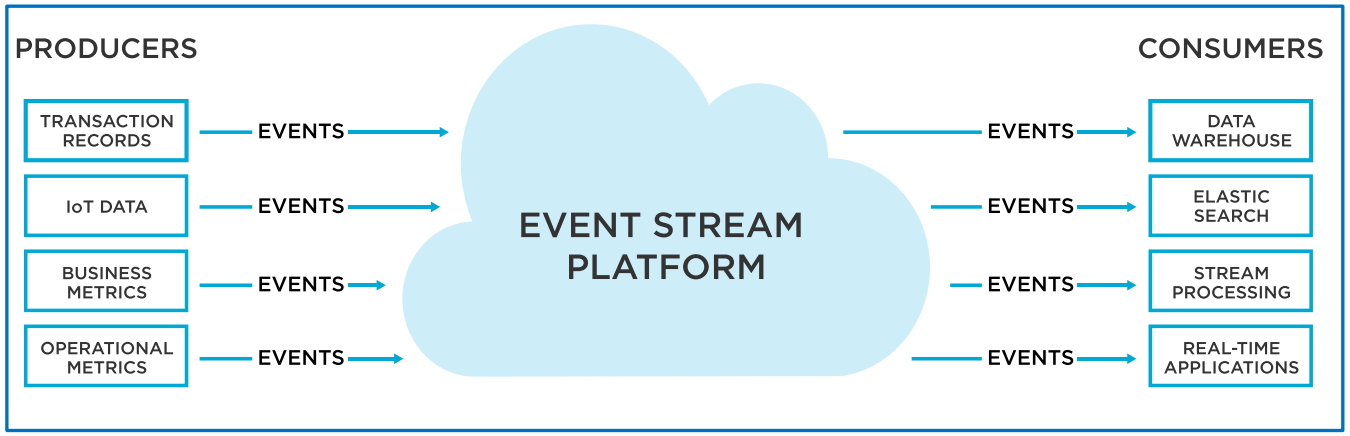
Technologies majeures :
- RabbitMQ
- Apache Kafka
- ActiveMQ

# Streaming d'évènements

Le streaming d'événements est un modèle dans lequel les événements générés par les systèmes (logs, transactions, clics, mesures, signaux…) sont capturés, stockés et traités en continu sous forme de flux (streams).

Contrairement aux systèmes classiques où les données sont stockées puis traitées plus tard, le streaming traite les données au moment où elles arrivent.

→ Les événements sont stockés dans un log append-only (journal en écriture seulement).




Dans ce modèle :

1. les producteurs publient des événements dans un flux,
2. les consommateurs lisent ces événements en continu,
3. les événements sont persistés dans un journal (log) distribué,
4. le traitement peut être temps réel ou quasi temps réel.

Technologies majeures :

- Kafka Streams
- Apache Pulsar
- Amazon Kinesis

# Mécanismes fondamentaux

## Acknowledgements (ACK / NACK)

l'acknowledgement (ACK) est un accusé de réception envoyé par le consommateur au broker pour confirmer qu'un message a bien été reçu et traité ; le NACK (negative ACK) indique un échec de traitement.
sans ACK, le broker ne sait pas si un message a été consommé correctement et risque de le perdre ou de le supprimer prématurément.

## Politique de réessai (Retry Policies) et backoff

→ Règles qui déterminent combien et quand retenter la livraison d'un message en cas d'échec.
- ✓ stratégie simple (n tentatives fixes)
- ✓ exponential backoff (attente croissante : t, 2t, 4t…)
- ✓ jitter (ajout d'aléa pour éviter les pics synchrones)

➔ Après un nombre maximal d'essais, le message est envoyé en Dead Letter Queue.

# Mécanismes fondamentaux

## Garanties d'ordre et partitionnement

L'ordering fixe les règles d'ordre de livraison et de traitement des messages, tandis que le partitionnement divise un flux en sous-flux pouvant être traités en parallèle.
Le partitionnement permet de gagner en scalabilité mais au prix de l'abandon d'un ordre global strict. On garantit alors seulement un ordre local à l'intérieur de chaque partition.

## Durabilité des messages (persistance)

Capacité à stocker les messages de façon persistante (disque, log) pour survivre aux pannes.
→ Pour éviter la perte d'informations en cas de crash du broker ou de la machine. 3 mécanismes utilisés :
messages en mémoire (volatile)
Messages persistants → écriture sur un support non-volatile (disque, log append-only)
Messages redondants → la duplication sur plusieurs nœuds indépendants pour assurer la tolérance aux pannes.

# Mécanismes fondamentaux

## Garanties de livraison des messages

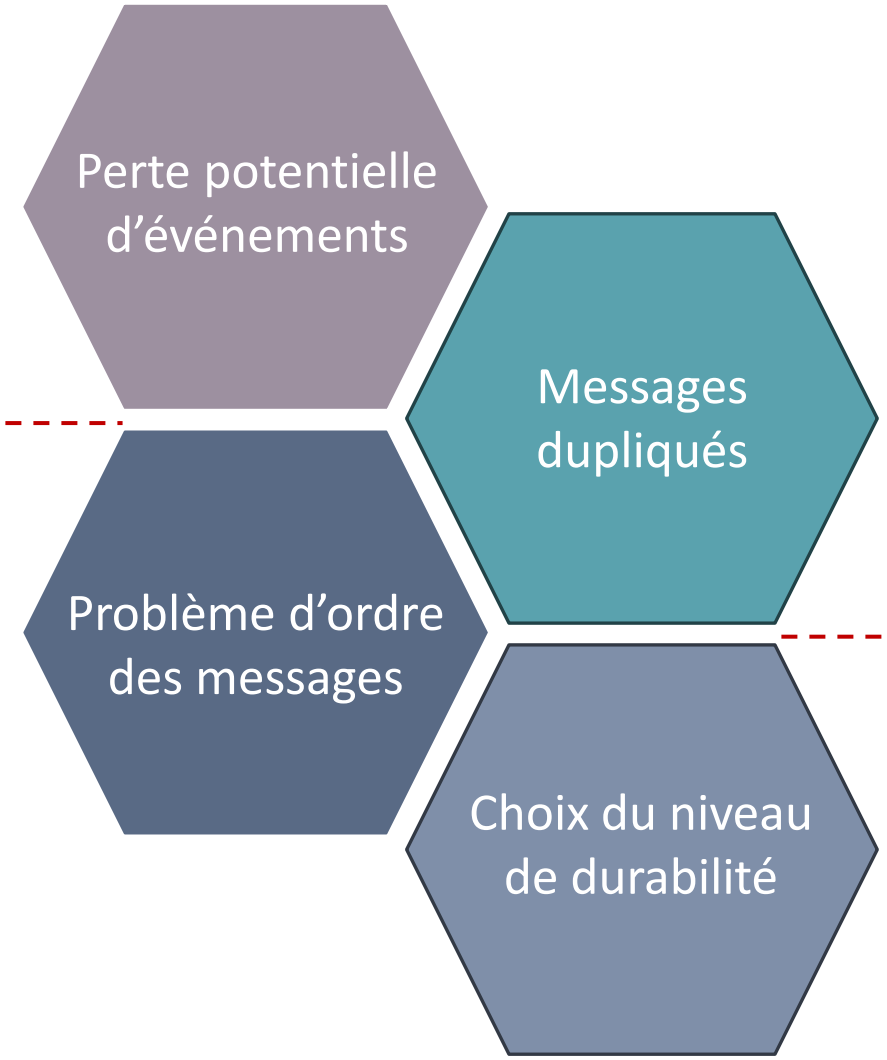
Les systèmes de communication asynchrone offrent différents niveaux de garantie sur la livraison des messages, selon le degré de fiabilité exigé par l'application.

- Au-plus-une fois (At-most-once) :
  Le message est envoyé sans mécanisme de réessai, donc il peut être perdu, mais il ne sera jamais livré deux fois. → perte tolérable mais duplication inacceptable.
  Exemple : Notification de logs
- Au-moins-une fois (At-least-once) :
  Le message est réémis tant qu'il n'a pas été confirmé (ACK), garantissant qu'il sera livré, mais il peut être livré plusieurs fois. → Le consommateur doit donc être idempotent.
  Exemple : Envoi d'un email de confirmation de commande.
- Exactement-une fois (Exactly-once) :
  Le système garantit que chaque message sera traité une seule fois, même en cas d'échec ou de redémarrage. → La garantie la plus forte mais aussi la plus coûteuse.
  Exemple : Traitement d'un paiement ou d'une transaction bancaire

# Défis de la communication asynchrone

Un message peut être perdu si le broker ou le producteur n'est pas correctement configuré pour garantir la persistance et les confirmations (ACK).

Les messages ne sont pas toujours reçus dans le même ordre que celui où ils ont été envoyés, surtout en présence de plusieurs partitions ou consommateurs.



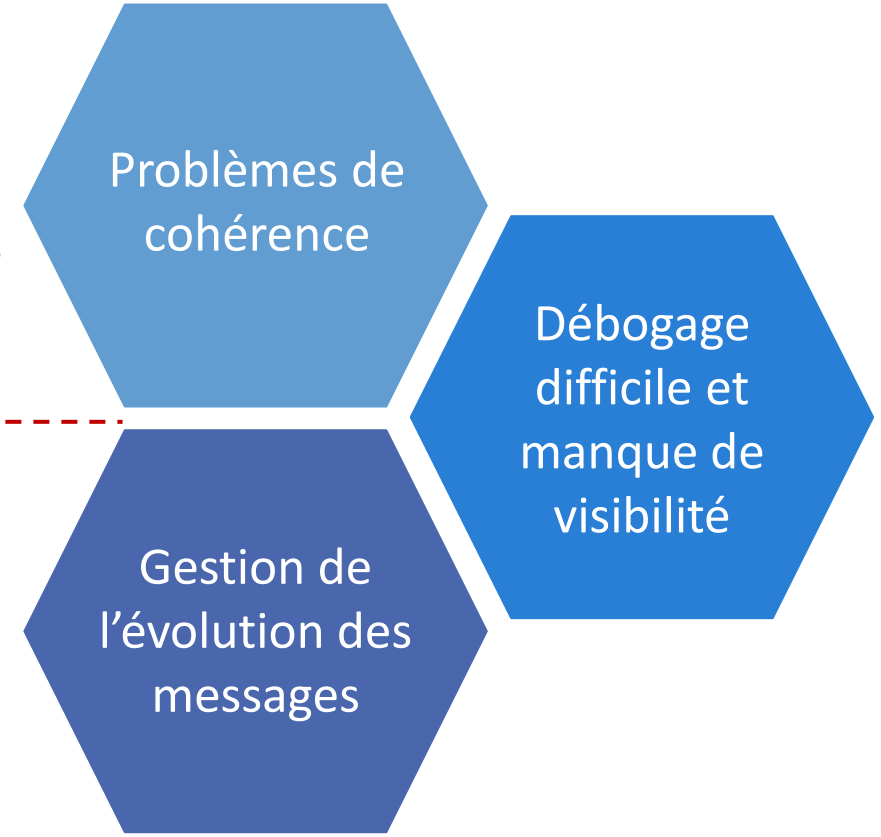

Comme la plupart des systèmes garantissent "au moins une fois", un même message peut être livré plusieurs fois, obligeant à rendre les traitements idempotents.

Il faut décider si les messages doivent être stockés en mémoire, sur disque ou répliqués, ce qui impacte directement la performance et la fiabilité.

# Défis de la communication asynchrone

Avec l'asynchronisme, les différents services peuvent voir des états temporaires différents, car les mises à jour ne sont pas instantanées.

Changer le format d'un message peut casser des consommateurs existants, nécessitant une gestion stricte des versions et de la compatibilité des schémas.




Comme un message traverse plusieurs services de façon décalée dans le temps, retracer l'origine d'un problème nécessite des outils d'observabilité avancés (logs, tracing, métriques).